

Kindo School Payments Challenge: Candidate Brief

Introduction

Thank you for your interest in joining Kindo as a Full Stack Engineer! This coding challenge is designed to give you an opportunity to showcase your skills with our tech stack while solving a problem similar to what you might encounter in this role.

The challenge focuses on building a simple school payment feature that interfaces with a legacy payment API. You'll create both a frontend interface and backend services to support payment collection for a school activity.

This is a chance for you to demonstrate both your engineering and ai skills. We recognise and want people to use the latest coding agents and assistants, but also know if those tools are hallucinating or going awry. Using AI and documenting how you used it is a requirement of this challenge (see **AI Workings** below).

We expect the time investment on this challenge to be around 2-4 hours. Please try to complete the challenge within 5 days of it being issued to you.

Let us know when you've completed the challenge so we can have time to assess it and then be prepared with a laptop to bring to the demo to us so you can run the application and dive into the code if needed.

Please reach out to us if you have any questions.

Problem Statement

A school needs to collect payments for an upcoming field trip. Parents need to be able to:

- View trip details (date, cost, location)
- Register their child for the trip
- Make a payment through an online form

- Receive confirmation of payment

You'll need to build a small full-stack application that handles this flow while integrating with our legacy payment processing API.

Technical Requirements

Backend (Python/FastAPI)

- Create a FastAPI service that:
  - Provides endpoints to fetch trip details
  - Accepts and validates payment submissions
  - Interfaces with the legacy payment API
  - Stores transaction records
  - Returns appropriate responses to the frontend
- Legacy API Integration:
  - You must integrate with our legacy payment processing API (details below)
  - The API expects a specific format and returns success/failure responses that you'll need to handle

Frontend (React)

- Create a small React application that:
  - Displays the field trip details
  - Includes a registration form for child's information
  - Provides a payment form with validation
  - Shows appropriate success/error messages
  - Has a clean, responsive design (Tailwind CSS encouraged but not required)

Legacy Payment API Interface

```
# This simulates the legacy payment API that you'll need to
integrate with# You should include this in your project and
```

```
call it as needed
import time
import random
from dataclasses import dataclass
from typing import Optional
@dataclass
class PaymentResponse:
    success: bool
    transaction_id: Optional[str] = None
    error_message: Optional[str] = None
class LegacyPaymentProcessor:
    """
    Legacy payment processor that accepts payments and prov
ides responses.
    This is a simulation of an external API you'd need to w
ork with.
    """

    def process_payment(self, payment_data):
        """
        Process a payment with the following required field
s:
        - student_name: str
        - parent_name: str
        - amount: float
        - card_number: str (must be 16 digits)
        - expiry_date: str (format: MM/YY)
        - cvv: str (must be 3 digits)
        - school_id: str
        - activity_id: str
        """
        required_fields = [
            'student_name', 'parent_name', 'amount',
            'card_number', 'expiry_date', 'cvv',
            'school_id', 'activity_id'
        ]

# Validate all required fields exist
```

```
        for field in required_fields:
            if field not in payment_data:
                return PaymentResponse(
                    success=False,
                    error_message=f"Missing required field:
{field}"
                )

# Validate card number (must be 16 digits)
        card_num = payment_data['card_number'].replace(' ',
'')
        if not (card_num.isdigit() and len(card_num) == 1
6):
            return PaymentResponse(
                success=False,
                error_message="Invalid card number. Must be
16 digits."
            )

# Validate expiry date format (MM/YY)
        if not self._validate_expiry_format(payment_data['e
xpiry_date']):
            return PaymentResponse(
                success=False,
                error_message="Invalid expiry date format.
Use MM/YY."
            )

# Validate CVV (must be 3 digits)
        if not (payment_data['cvv'].isdigit() and len(payme
nt_data['cvv']) == 3):
            return PaymentResponse(
                success=False,
                error_message="Invalid CVV. Must be 3 digit
s."
            )

# Validate amount is positive
```

```
        if not isinstance(payment_data['amount'], (int, flo
at)) or payment_data['amount'] <= 0:
            return PaymentResponse(
                success=False,
                error_message="Payment amount must be a pos
itive number."
            )

# Simulate processing time
        time.sleep(1.5)

# Simulate occasional payment failures
        if random.random() < 0.1:# 10% chance of failure
            return PaymentResponse(
                success=False,
                error_message="Payment declined by processo
r. Please try again."
            )

# Generate a transaction ID for successful payments
        transaction_id = f"TX-{int(time.time())}-{random.ra
ndint(1000, 9999)}"

        return PaymentResponse(
            success=True,
            transaction_id=transaction_id
        )

    def _validate_expiry_format(self, expiry):
        """Validate the expiry date is in MM/YY format."""
        if not isinstance(expiry, str) or len(expiry) != 5:
            return False

        if expiry[2] != '/':
            return False

        month, year = expiry.split('/')
```

```
        if not (month.isdigit() and year.isdigit()):
            return False

        month_num = int(month)
        if month_num < 1 or month_num > 12:
            return False

        return True
```

Constraints/Requirements

- **Tech Stack**:
  - Backend: Python with FastAPI
  - Frontend: React
  - Relational Database: Whatever free relational db you prefer, e.g. SQLite is fine for this challenge
- **Must-Have Features**:
  - Functional payment flow from frontend to backend
  - Integration with the legacy payment API
  - Form validation on both frontend and backend
  - Error handling and user feedback
  - At least basic styling for a good user experience
  - Documented AI workings (see **AI Workings** below)
- **Nice-to-Have Features**:
  - Responsive design with Tailwind CSS
  - Unit tests for critical components
  - Error logging
  - Loading states during payment processing
  - Receipt generation
- **Evaluation Criteria**:
  - Code quality and organization (including regular commits)

- Problem-solving approach
- Error handling
- User experience considerations
- Documentation
- How you use AI tools, and how you check and take ownership of their output

AI Workings (Required)

We are an AI-forward team and expect you to use AI coding agents and assistants (e.g. Claude Code, Cursor, GitHub Copilot, ChatGPT) for this challenge. We are interested in how you use them, not just what they produce. Include an `AI_WORKINGS.md` file in your repository covering:

- Which tools you used, and what for
- Your key prompts or instructions, plus any agent config files (e.g. `CLAUDE.md`, `AGENTS.md`, Cursor rules) committed to the repo
- At least one time the AI got something wrong or went off track, how you noticed, and what you did about it
- Anything you chose to write, rewrite or reject yourself, and why

Rough notes are fine. We value honesty over polish. Be ready to talk us through this at the demo.

Bonus Points

- **Deployment**: Deploy your solution to a free hosting service (Vercel, Netlify, Heroku, AWS Free Tier, etc.)
- **React** : Include a simple responsive web view to support mobile devices.

Submission Guidelines

- Create a GitHub/BitBucket repository for your solution
- Include a README with:
  - Setup instructions
  - Overview of your solution

    - Any assumptions or design decisions
    - A link to your `AI_WORKINGS.md`
- Be prepared to walk through your code and explain your decisions during the interview

We're excited to see your approach to this challenge! Remember, we're not just looking for a working solution but also evaluating your coding style, problem-solving skills, and how you approach integrating with legacy systems.

Good luck!